

Introduction to Computing (ITC)

CS1001

Lecture-9

Programming in C

Elements of Computer Program

1. Identifiers: **Variable, Constant** Examples: (x, y, z, sum, var, temp)

2. Data Types: int, float, char Examples: (int x, float y, char z)

3. Statement: Examples: printf(“%d”,x);
printf(“hello world”);

4.1. **Assignment** (sum=x+y, x=y, sum=0)

4.2. **Expression**

Arithmetic (x+y)

Logical (x && y)

Relational (x<y)

5. Control Structures

- Sequential

- Conditional (if, switch case)

- loop/Iteration (while do, do while, for)

Identifiers

- Identifiers are the names used to identify variables, functions, arrays, structures, or any other user-defined things.

```
int value = 10;
```

```
//creating a variable
```

```
void create() {  
}
```

```
//creating a function
```

Here **value** and **create** are identifiers

Rules for Identifiers

- Identifier can contain following characters:
 - Uppercase (A-Z) and lowercase (a-z) alphabets.
 - Numeric digits (0-9).
 - Underscore (_).
- The first character of an identifier must be a **letter** or an **underscore**.
- Case-sensitive
- Cannot be keywords in C (such as int, return, if, while, etc.).

Types of identifiers

- Name of
 - Variables
 - Functions
 - Arrays
 - Pointers
 - Many more.....

Valid and Invalid Identifiers

- Valid identifiers

X

abc simple_interest

a123

LIST

stud_name

Empl_1 Empl_2

avg_empl_salary

- Invalid identifiers

10abc my-

name

“hello”

simple interest

(area)

%rate

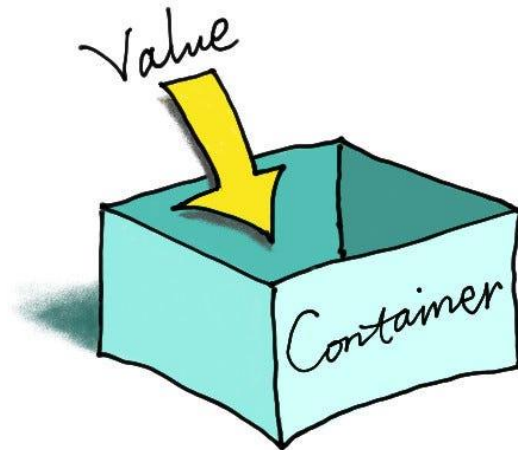
C Keywords

- Set of reserved words which cannot be use for identifiers.
- Normally 32 keywords in C

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
continue	for	signed	void
do	if	static	while
default	goto	sizeof	Volatile
const	float	short	Unsigned

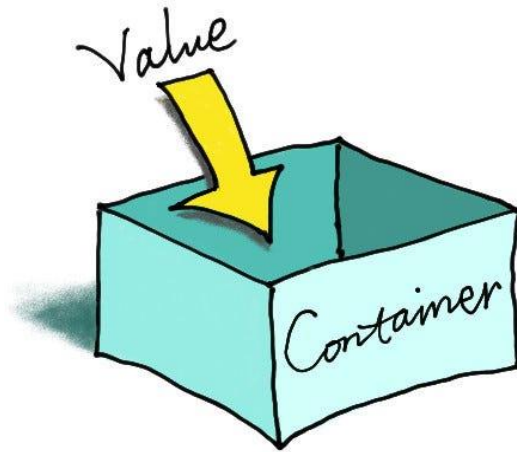
What is a Variable?

- A variable is an identifier which is used to store some value
- Can store any temporary result while executing a program



Variables

- Variables are stored in memory
- Remember that memory is a list of storage locations, each having a unique address



Values for Variable

- Can have only one value assigned to it at any given time during the execution of the program
- The value of a variable can be changed during the execution of the program
- A variable is like a **bin**
 - The contents of the bin is the **value** of the variable
 - The variable name is used to refer to the value of the variable
 - A variable is mapped to a location of the memory called its **address**

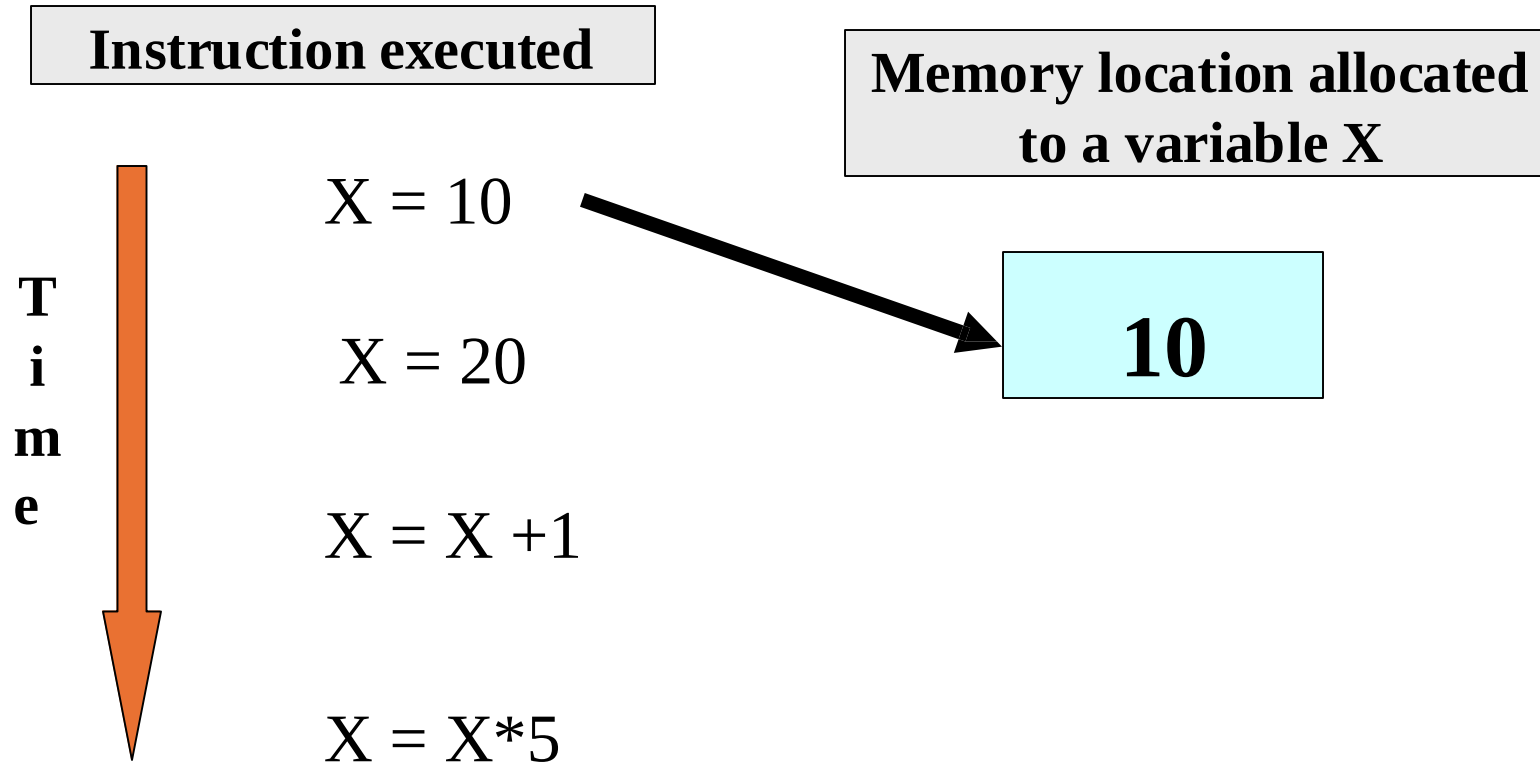
Rules for defining Variable Names

- Sequence of letters and digits
 - First character must be a letter or '_'
 - No special characters other than '_'
 - No space in between
- Names are **case-sensitive** (**mahindra** and **Mahindra** are two different names)
- Examples of valid names:
 - **i j rank1MAX max Min it_rank**
- Examples of invalid names:
 - it's 2sqrt int ratio%**

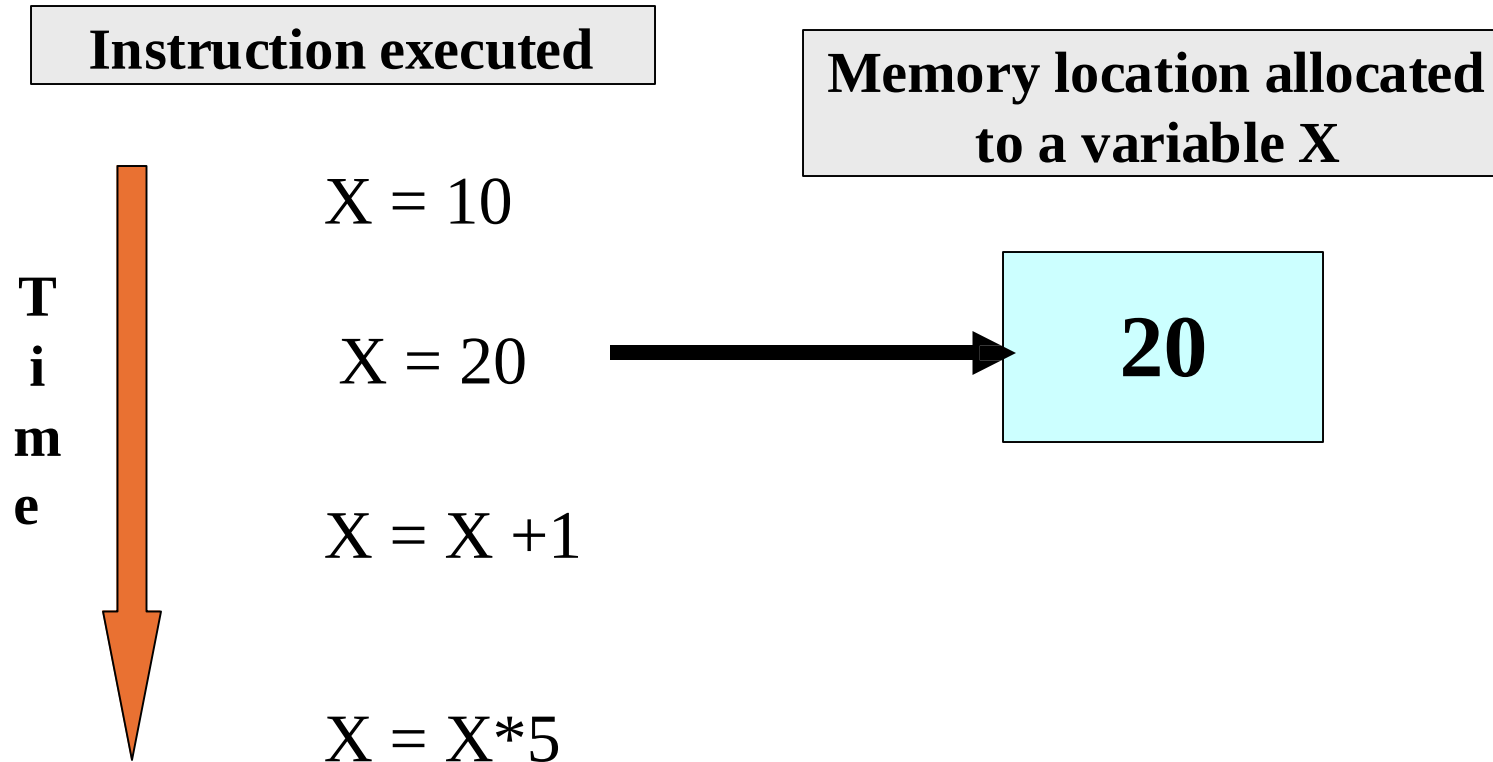
Sample code

```
#include <stdio.h>
void main( )
{
    int x;
    int y;
    x=1;
    y=3;
    printf("x = %d, y= %d\n", x, y);
}
```

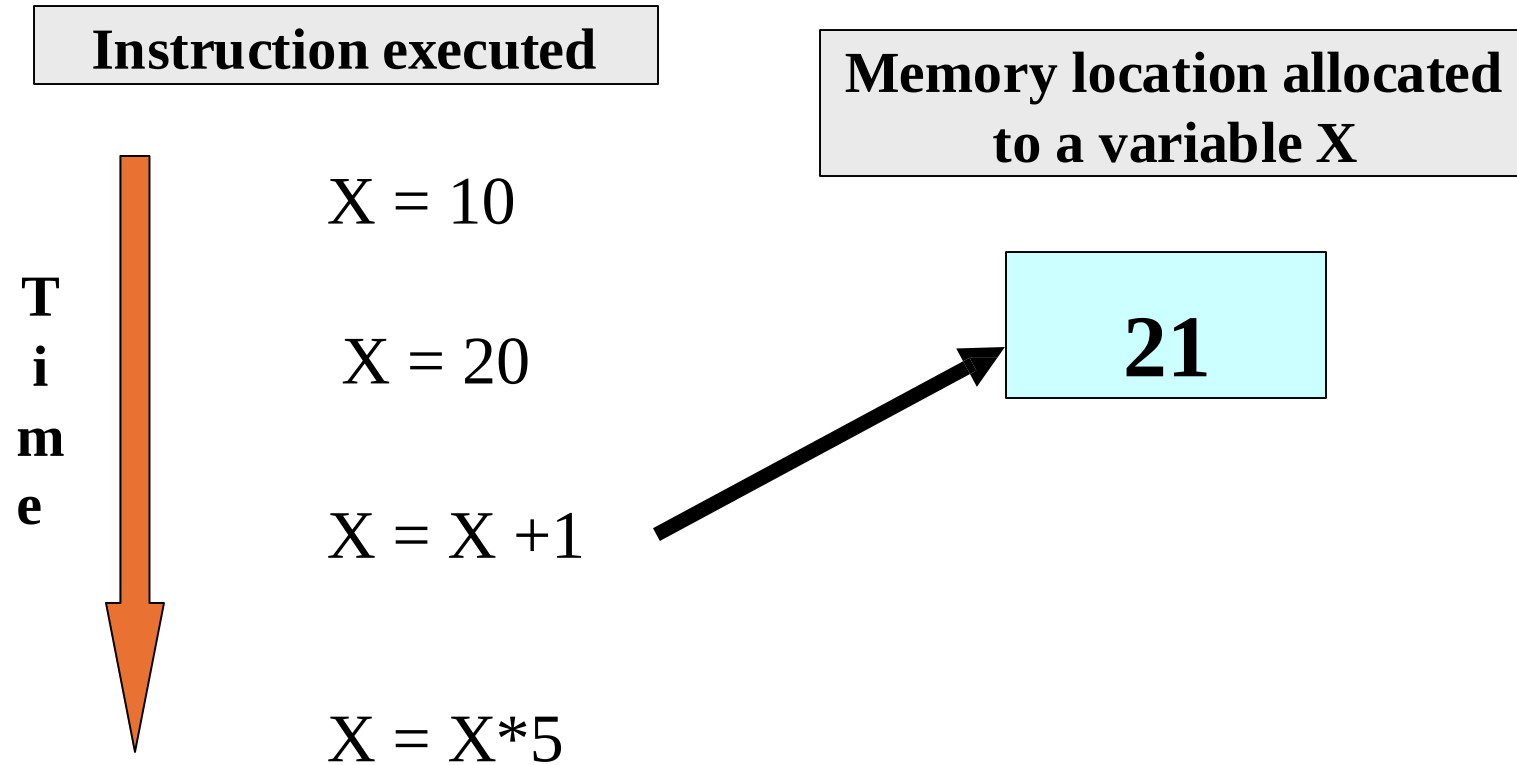
Variables in Memory



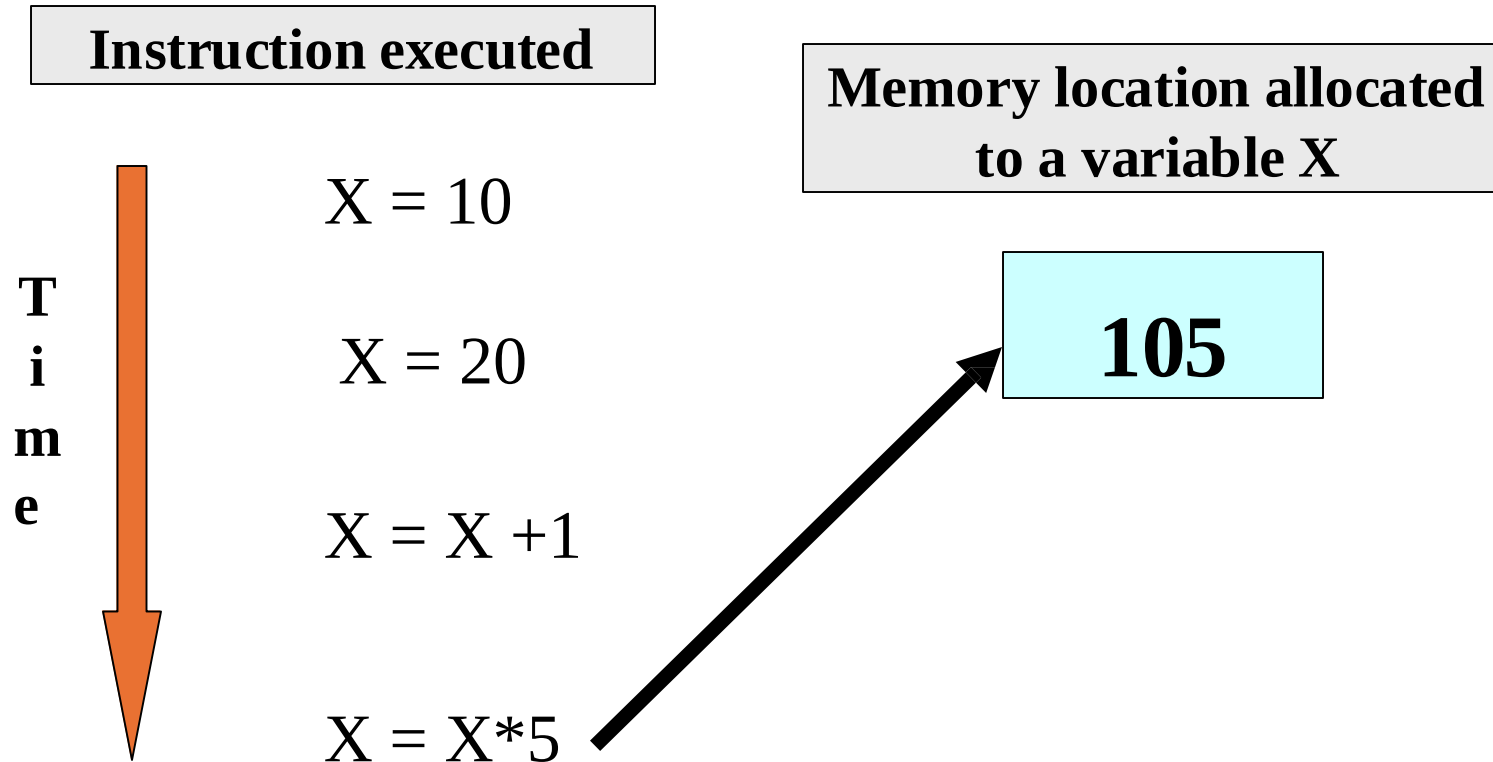
Variables in Memory



Variables in Memory



Variables in Memory



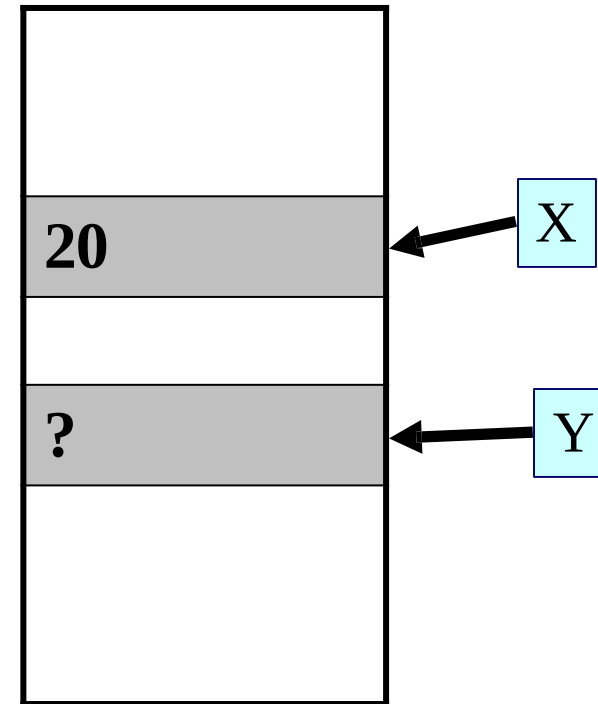
Variables

$X = 20$

$Y = 15$

$X = Y + 3$

$Y = X / 6$



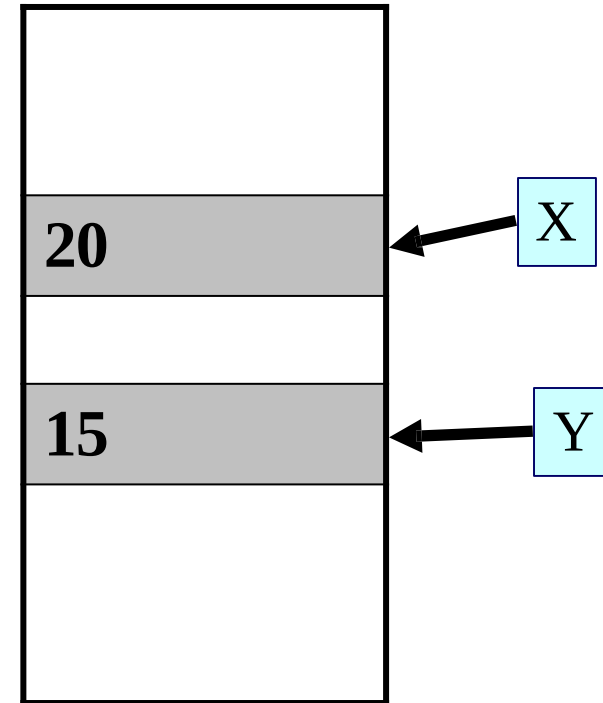
Variables

$X = 20$

$Y = 15$

$X = Y + 3$

$Y = X / 6$



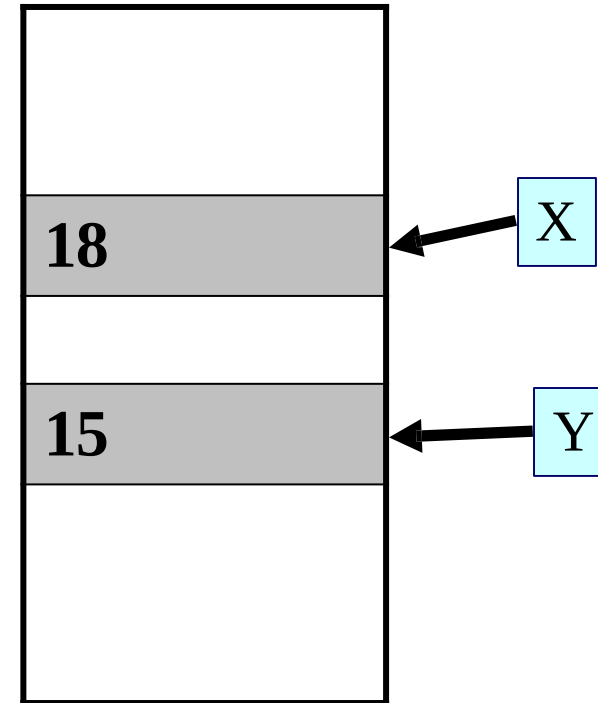
Variables

$X = 20$

$Y = 15$

$X = Y + 3$

$Y = X / 6$



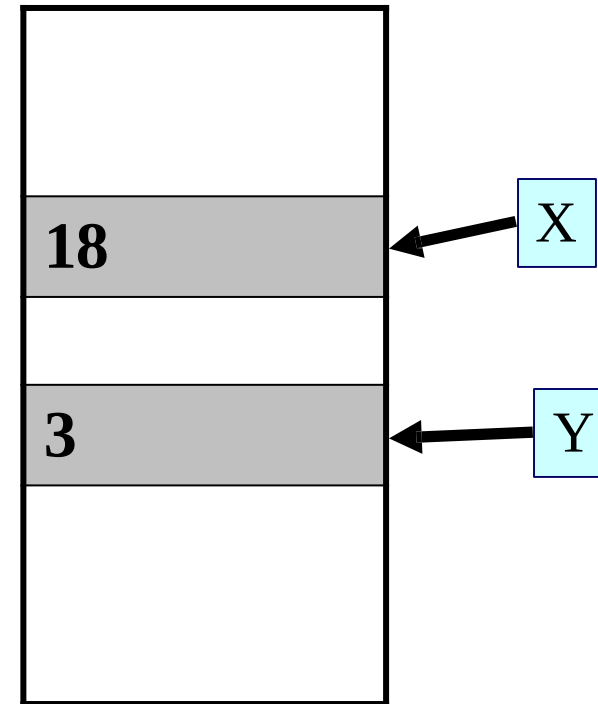
Variables

$X = 20$

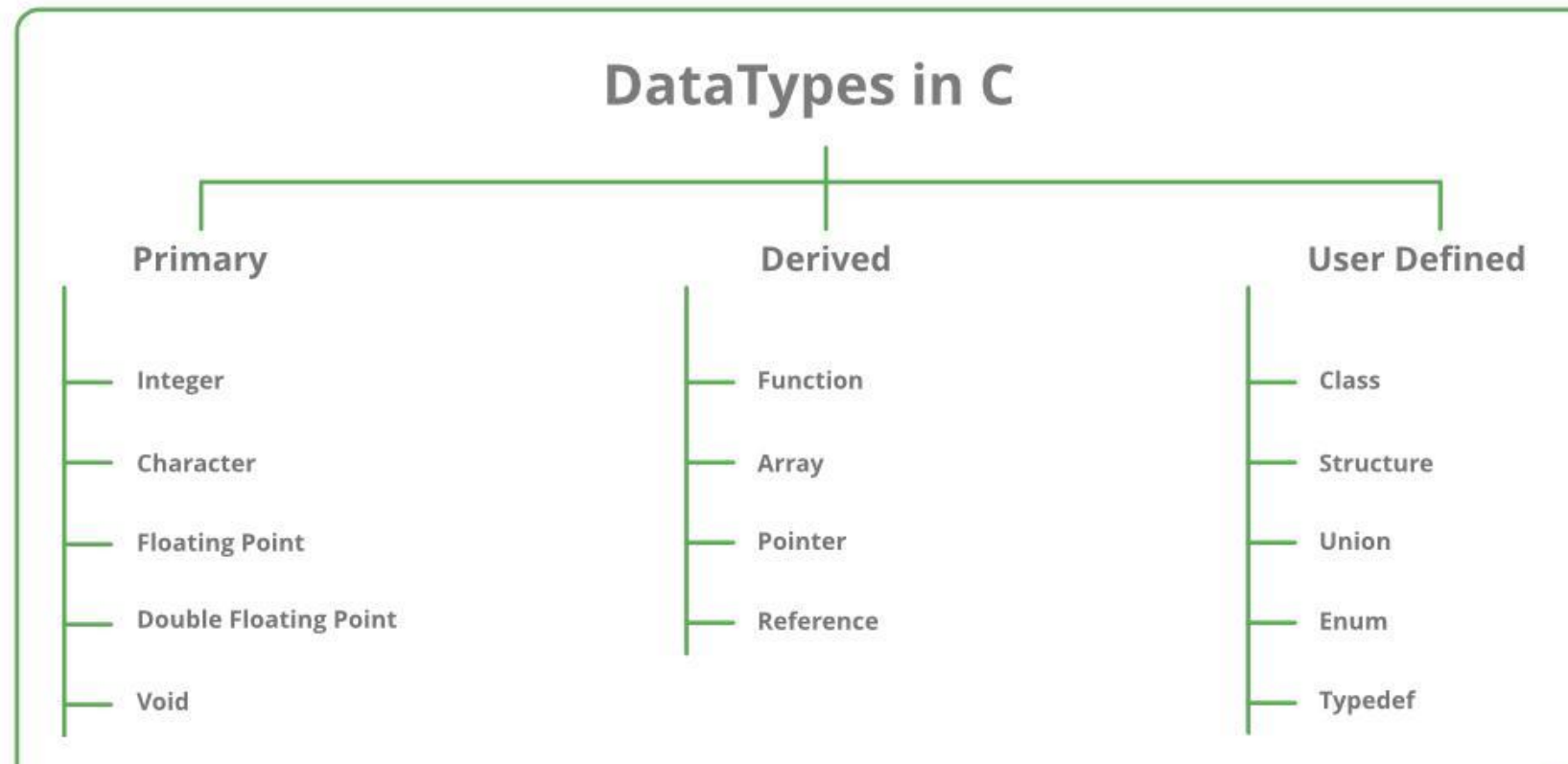
$Y = 15$

$X = Y + 3$

$Y = X / 6$



Datatypes in C



Data Types

- ❖ Each **variable** has a **type** which indicates what type of values the variable can hold
- ❖ Four common data types in C
 - ❖ **int** - can store integers (usually 4 bytes)
 - ❖ **float** - can store single-precision floating point numbers (usually 4 bytes)
 - ❖ **double** - can store double-precision floating point numbers (usually 8 bytes)
 - ❖ **char** - can store a character (1 byte)

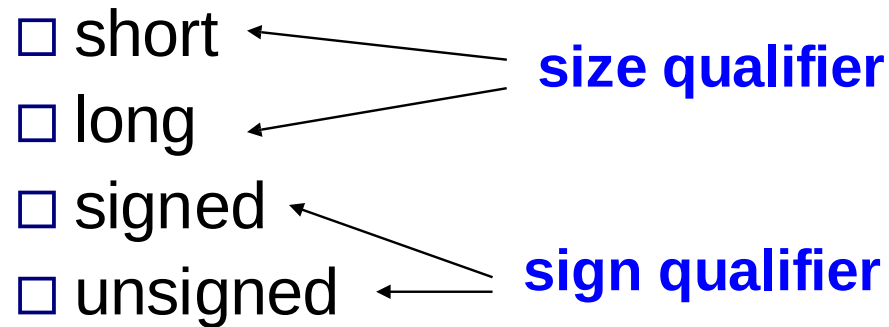
Data types

- Must declare a variable (specify its **type** and **name**) before using it anywhere in your program
- All variable declarations should be at the beginning of the `main()` or other functions
- A value can also be assigned to a variable at the time the variable is declared.

```
int  speed = 30; char  
    flag = 'y';
```

More Data Types in C

- Some of the basic data types can be augmented by using certain data type qualifiers:



- Typical examples:
 - ☐ short int (usually 2 bytes)
 - ☐ long int (usually 4 bytes)
 - ☐ unsigned int (usually 4 bytes, but no way to store + or -)

Some data types

(some of these can vary depending on type of machine)

Integer data type	Bit size	Minimum value	Maximum value
char	8	$-2^7 = -128$	$2^7 - 1 = 127$
short int	16	$-2^{15} = -32768$	$2^{15} - 1 = 32767$
int	32	$-2^{31} = -2147483648$	$2^{31} - 1 = 2147483647$
long int	32	$-2^{31} = -2147483648$	$2^{31} - 1 = 2147483647$
long long int	64	$-2^{63} = -9223372036854775808$	$2^{63} - 1 = 9223372036854775807$
unsigned char	8	0	$2^8 - 1 = 255$
unsigned short int	16	0	$2^{16} - 1 = 65535$
unsigned int	32	0	$2^{32} - 1 = 4294967295$
unsigned long int	32	0	$2^{32} - 1 = 4294967295$
unsigned long long int	64	0	$2^{64} - 1 = 18446744073709551615$

Example 1

```
#include <stdio.h>
```

```
void main()
```

```
{
```

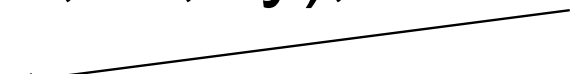
```
    int x, y, sum;
```

Three int type variables declared



```
    scanf("%d%d",&x,&y);
```

Values assigned



```
    sum = x + y;
```

```
    printf( "%d plus %d is %d\n", x, y, sum );
```

```
}
```

Example 2

```
#include <stdio.h>
void main()
{
    float x, y;
    int d1, d2 = 10;
    scanf("%f%f%d",&x, &y, &d1);
    printf( "%f plus %f is %f\n", x, y, x+y);
    printf( "%d minus %d is %d\n", d1, d2, d1-d2);
}
```

Assigns an initial value to d2,
can be changed later



Read-only variables (const keyword)

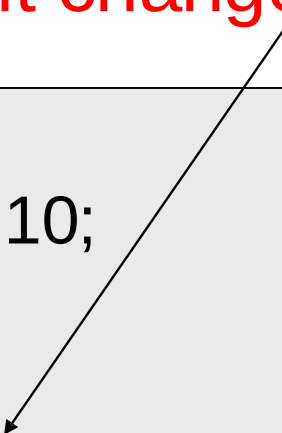
- Variables whose values can be initialized during declaration, but cannot be changed after that
- Declared by putting the `const` keyword in front of the declaration
- Storage allocated just like any variable
- Used for variables whose values need not be changed

Correct

```
void main() {  
    const int LIMIT = 10;  
    int n;  
    scanf("%d", &n);  
    if (n > LIMIT)  
        printf("Out of limit");  
}
```

Incorrect: **Limit changed**

```
void main() {  
    const int Limit = 10;  
    int n;  
    scanf("%d", &n);  
    Limit = Limit + n;  
    printf("New limit is %d", Limit);  
}
```



Constants

■ Integer constants

- Consists of a sequence of digits, with possibly a plus or a minus sign before it
- Embedded spaces, commas and non-digit characters are not permitted between digits

Which of the following are valid and invalid short integer constants?

52 ✓	86142 ✗	32.8 ✗
0.96 ✗	92 ✓	
5,200 ✗		

Short int is 16-bit

Max No
32767

Constants

e means "10 to the power of"

■ Floating point constants

■ 32.68 ✗ 5.8 ✗ 76.91 ✗

By default
double
32.68 f
5.8 f
76.91 f } -> float

■ Double constants

Two different notations:

□ Decimal notation: 25.0, 0.0034, .84, -2.234

□ Exponential (scientific) notation

3.45e23, 0.123e-12, 123e2

□ Long doubles : 52ld ✗ 52.0ld ✓

ld → long double

Constants

■ Character constants

- Contains a single character enclosed within a pair of single quote marks.
- Examples : '2', '+', 'Z'
- '\b', 32, 'a', '\o', 9

• String constants

Range -128 to 127

A = 65
a = 97

32 lies in range of char constants

"a"

inside " " quotes we write string

Backslash characters

- Some special backslash characters

`'\n'` new line

`'\t'` horizontal tab

`'\''` single quote

`'\"'` double quote

`'\\'` backslash

`'\0'` null

Input: **scanf** function

- Performs input from keyboard
- It requires a format string and a list of variables into which the value received from the keyboard will be stored
- format string = individual groups of characters

(usually ‘%’ sign, followed by a conversion character),
with one character group for each variable in the list

```
int a, b;  
float c;  
scanf("%d %d %f", &a, &b, &c);
```

**Variable list (note the &
before a variable name)**

Format string

Commonly used conversion characters

c	for char type variable
d	for int type variable
f	for float type variable
lf	for double type variable

Examples

```
scanf ("%d", &size) ;  
scanf ("%c", &nextchar) ;  
scanf ("%f", &length) ;  
scanf ("%d%d", &a, &b);
```

Reading a single character

- A single character can be read using `scanf` with `%c`
- It can also be read using the `getchar()` function

```
char c;  
c = getchar();
```

- Program waits at the `getchar()` line until a character is typed, and then reads it and stores it in `c`

Output: `printf` function

- Performs output to the standard output device (typically defined to be the screen)
- It requires a format string in which we can specify:
 - The text to be printed out, if num=125
 - Specifications on how to print the values
`printf ("The number is %d\n", num);`
 - The format specification `%d` causes the value listed after the format string to be embedded in the output as a decimal number in place of `%d`
 - Output will appear as: `The number is 125`

Output: `printf` function

- General syntax:

 - `printf (format string, arg1, arg2, ..., argn);`

 - format string refers to a string containing formatting information and data types of the arguments to be output

 - the arguments `arg1, arg2, ...` represent list of variables/expressions whose values are to be printed

- The conversion characters are the same as in `scanf`

- Examples:

```
printf ("Average of %d and %d is %f", a, b, avg);  
printf ("Hello \nGood \nMorning \n");  
printf ("%3d %3d %5d", a, b, a*b+2);  
printf ("%7.2f %5.1f", x, y);
```

Code & check!

- Many more options are available for both printf and scanf

- ☐ Read from the book and lab tutorials
- ☐ Practice them in the lab

Let's code and check

```
#include <stdio.h>
void main()
{
    printf ("Hello, World! ");
    printf ("Hello \n World! \n");
}
```

Some more printing

```
#include <stdio.h>

void main()
{
    printf ("Hello, World! \n") ;
    printf ("Hello \n World! \n") ;
    printf ("Hell\no \t World! \n") ;
}
```

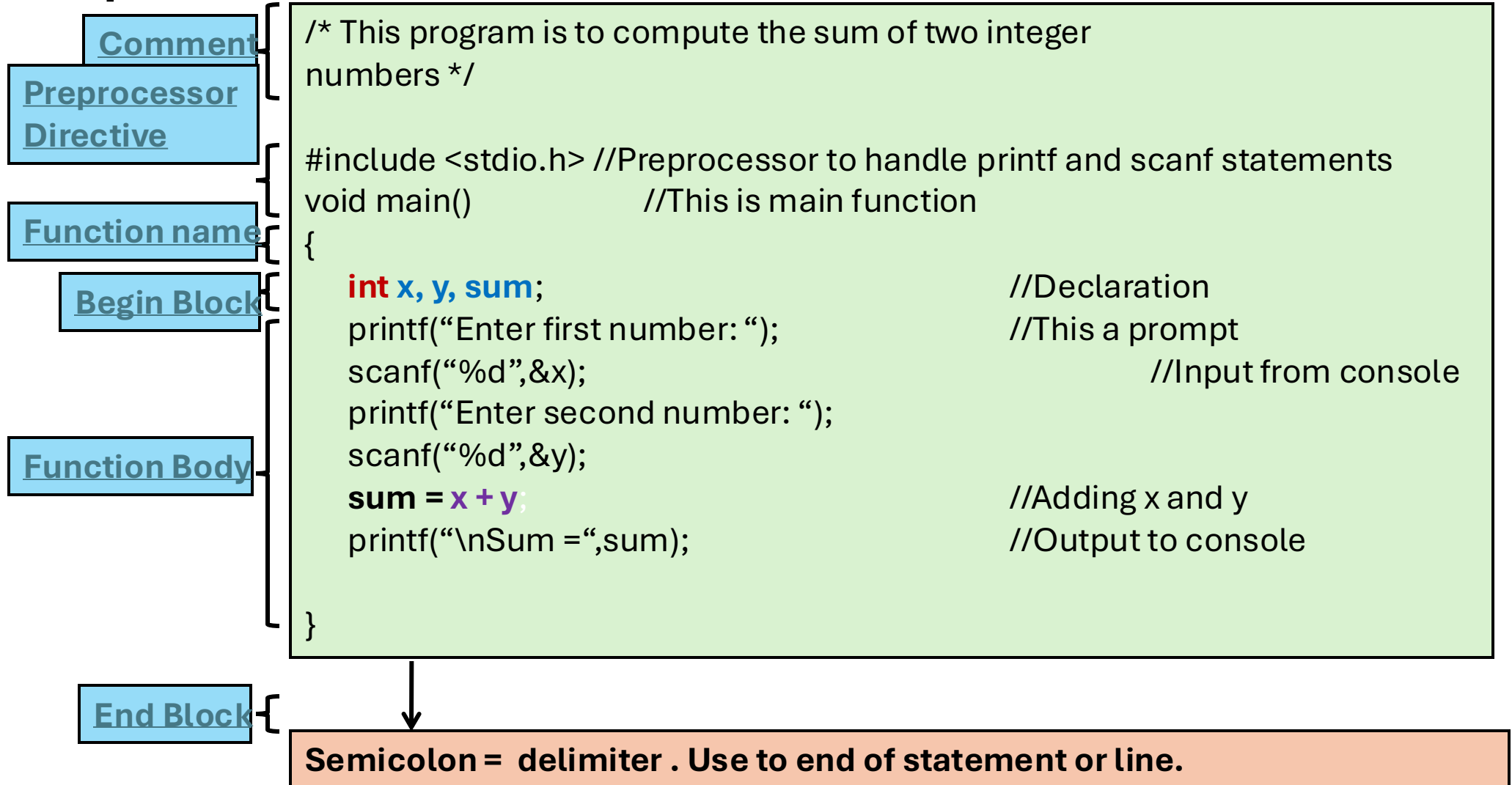
Reading values from keyboard

```
#include <stdio.h>
void main()
{
    int num ;
    scanf ("%d", &num) ;
    printf ("No. of students is %d\n", num) ;
}
```

Structure of a C program

- A collection of **functions** (we will see what they are later)
- Exactly one special function named **main** must be present. Program always starts from there
- Each function has statements (instructions) for declaration, assignment, condition check, looping etc.
- Statements are executed one by one

Example for C code



C character set

The C language alphabet

- Uppercase letters 'A' to 'Z'
- Lowercase letters 'a' to 'z'
- Digits '0' to '9'
- Certain special characters:

!	#	%	^	&	*	(	)
-	_	+	=	~	[	]	\
	;	:	'	"	{	}	,
.	<	>	/	?	blank		

A C program should not contain anything else

What is a Good Program?

A good program is a program that is easy to read and easy to be understood by other programmers...

Good Programming Styles

Spacing – each instruction begin with new line

Indent – each instruction has indention depends on it structure

Comment – help a human reader to understand the program

Variable name – use descriptive name

Rules of naming the variable:

- Other than reserved word (do not use char, int, float, double, long, String etc..)
- Alphanumeric A-Z, a-z, 0-9, - (underscore) but do not use number (0-9) as variable.
- Length recommended 3-8 characters

Spacing

- Each instruction should begin with new line(s)

//Good

```
#include <stdio.h>
void main()
{
    float x, y;
    int d1, d2 = 10;
}
```

//Bad

```
#include <stdio.h>
void main(){    float x, y; int d1, d2 = 10; }
```

Indent

```
//Good
#include <stdio.h>
    void main()
    {
        float x, y;
        int d1, d2 = 10;
    }
```

//Bad

```
//Bad
#include <stdio.h>
void main()
{
    float x, y;
    int d1, d2 = 10;
}
```

Comments

- Comment is used to explain the function of the program or statement in order to help them or others read and understand the programs or statement.
- Use two different punctuation marks
 - Begin with `/*` and end with `*/` - use if the comments runs more than one line.
 - `//` - use if takes a line or less
- Anything placed after the `//` or inside `/* */` are considered comments and will not be executed by the computer.

Comments and Meaningful variable names

```
//This program calculates age

main()
{
    int age;
    printf("Please enter year born: ");    //input year born
    scanf("%d",&yearBorn);
    age = curYear - yearBorn;            //calculate age
    printf("Your age is%d",age);    //output age to console
}
```

Centigrade to Fahrenheit

```
#include <stdio.h>
void main()
{
    float cent, fahr;
    scanf("%f",&cent);
    fahr = cent*(9.0/5.0) + 32;
    printf( "%f C equals %f F\n", cent, fahr);
}
```

Largest of two numbers

```
#include <stdio.h>
void main()
{
    int x, y;
    scanf("%d%d",&x,&y);
    if (x>y)
        printf("Largest is %d\n",x);
    else
        printf("Largest is %d\n",y);
}
```



Question 1

Before execution compiler converts high-level program into machine language program? This machine program is called

(a) Source program (b) Object program (c) Exe program (d) None

Question 2

C-Language is used for

- (a) System programming
- (b) Application programming
- (c) Both (a) and (b)
- (d) Neither (a) nor (b)

Question 3

Among the following which one is not a valid identifier?

Y12, Sum-1, _temperature, 4th

(a) Y12

(b) Sum-1

(c) _temperature

(d) 4th

Question 4

From the following determine the invalid character constant 'a', '\\', '\n', /n'

(a) 'a' (b) '\\ ' (c) '\n' (d) '/n'

Question 5

Which one is not C keyword?

(a) auto

(b) extern

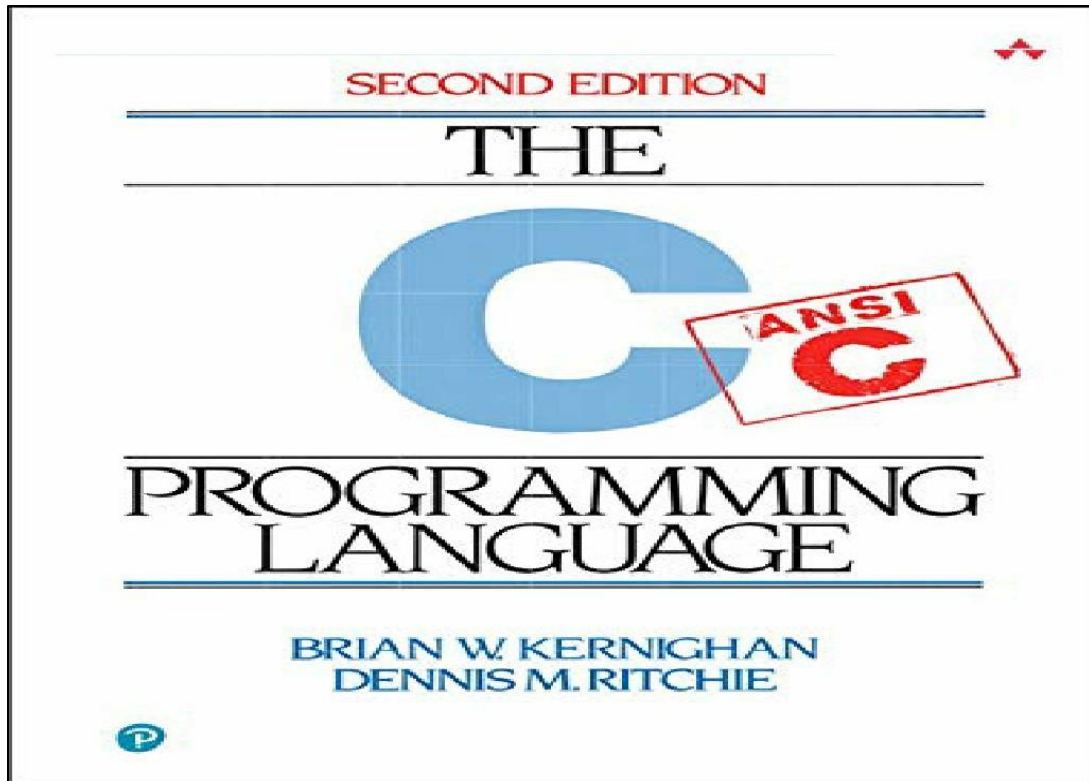
(c) register

(d) Main

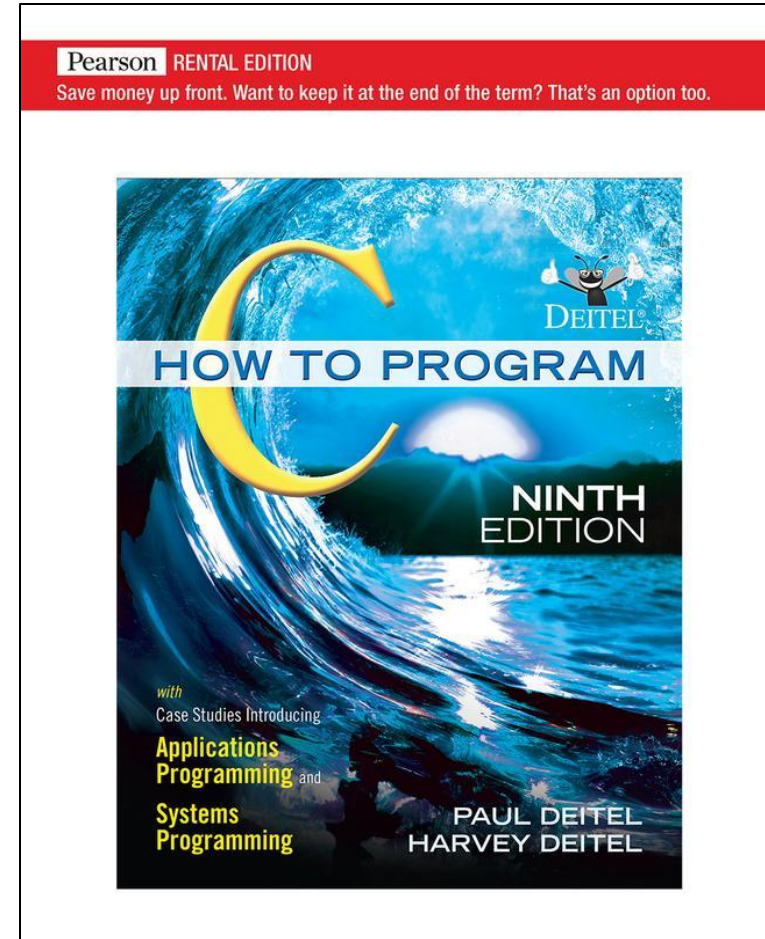
Acknowledgements

- Thanks to the Course and instructors [COL100 lecture notes: 20-21 Sem 2](#).
- Some slides has been adapted from topics available here.

Reference books/ materials



“The C Programming Language” by Brian Kernighan and Dennis Ritchie



“C How to Program” by Paul Deitel and Harvey Deitel